

Functional Programming Solutions

These are worked solutions to the Functional Programming Exercises. Each page completes the corresponding starting-point template, with a short explanation of the approach taken.

EXERCISE 1 - SOLUTION

Pure Functions & Avoiding Side Effects

Objective

Practice writing a pure function, and refactor an impure function so it no longer mutates its input.

Explanation

`calculate_total()` only reads its arguments and returns a new value, so it is pure. `apply_discount()` is fixed with a list comprehension, which builds a brand new list instead of writing into the caller's list, so the original list is left untouched.

```
● ● ● solution_1_pure_functions.py
def calculate_total(prices, tax_rate):
    return sum(prices) * (1 + tax_rate)

def apply_discount(prices, discount):
    return [p * (1 - discount) for p in prices]

if __name__ == "__main__":
    print(calculate_total([10, 20], 0.1))    # 33.0

    original = [100, 200]
    discounted = apply_discount(original, 0.1)
    print(discounted)    # [90.0, 180.0]
    print(original)      # [100, 200] (unchanged)
```

EXERCISE 2 - SOLUTION

Lambda Functions

Objective

Practice writing small lambda functions and using them as sort keys and predicates.

Explanation

get_score picks out the second element of each tuple, so sorted() can rank students by score.

reverse=True puts the highest score first. is_pass is a one-line predicate: it simply returns whether score is at least 70.

```
solution_2_lambda_functions.py

students = [("Amy", 82), ("Bo", 67), ("Cy", 91), ("Dee", 74)]

get_score = lambda s: s[1]
ranked = sorted(students, key=get_score, reverse=True)
is_pass = lambda score: score >= 70

if __name__ == "__main__":
    print(ranked)
    # [('Cy', 91), ('Amy', 82), ('Dee', 74), ('Bo', 67)]

    print(is_pass(74))    # True
    print(is_pass(67))    # False
```

EXERCISE 3 - SOLUTION

Transforming and Filtering with map() and filter()

Objective

Use map() and filter() to transform and select data without writing a manual for-loop.

Explanation

map() applies the Celsius-to-Fahrenheit formula to every value in one pass. filter() keeps only the values where the predicate (c > 20) is True. Both return lazy iterators, so list() is used to see the results.

```
solution_3_map_filter.py

temperatures_c = [0, 21, 33, -5, 40, 15]

temperatures_f = map(lambda c: c * 9/5 + 32, temperatures_c)
warm_days = filter(lambda c: c > 20, temperatures_c)

if __name__ == "__main__":
    print(list(temperatures_f))
    # [32.0, 69.8, 91.4, 23.0, 104.0, 59.0]

    print(list(warm_days))
    # [21, 33, 40]
```

EXERCISE 4 - SOLUTION

Aggregating Data with reduce()

Objective

Use `functools.reduce` to fold a sequence of items down to a single aggregated value.

Explanation

The first `reduce()` accumulates a running total, starting from 0. The second `reduce()` has no initial value, so it starts from the first basket item and keeps whichever of the two items being compared has the higher price.

```
from functools import reduce

basket = [
    {"item": "bread", "price": 2.5},
    {"item": "milk", "price": 1.8},
    {"item": "eggs", "price": 3.2},
]

total_price = reduce(lambda acc, item: acc + item["price"], basket, 0)
most_expensive = reduce(
    lambda a, b: a if a["price"] > b["price"] else b, basket
)

if __name__ == "__main__":
    print(round(total_price, 2))      # 7.5
    print(most_expensive["item"])    # eggs
```

EXERCISE 5 - SOLUTION

Higher-Order Functions & functools.partial

Objective

Build a higher-order function and use `functools.partial` to pre-configure it.

Explanation

`convert_currency()` is a general-purpose function that needs both an amount and a rate. `partial()` locks in the rate for each currency, producing `to_usd` and `to_gbp`: ordinary-looking functions that only still need an amount.

```
from functools import partial

def convert_currency(amount, rate):
    return amount * rate

to_usd = partial(convert_currency, rate=1.09)
to_gbp = partial(convert_currency, rate=0.79)

if __name__ == "__main__":
    print(round(to_usd(100), 2))    # 109.0
    print(round(to_gbp(100), 2))    # 79.0
```

EXERCISE 6 - SOLUTION

Composing a Functional Pipeline

Objective

Combine map, filter and reduce into a pipeline, then reproduce the same result using pandas.

Explanation

The pipeline filters out negative orders, applies the 5% charge with map, then reduce sums what is left. The pandas version reaches the same total with apply(): the lambda returns 0 for negative orders so they contribute nothing to the sum.

```
from functools import reduce
import pandas as pd

orders = [12, -4, 30, 8, -2, 19]

pipeline_total = reduce(
    lambda acc, n: acc + n,
    map(lambda n: n * 1.05, filter(lambda n: n > 0, orders)),
    0
)

df = pd.DataFrame({"order": orders})
df["charged"] = df["order"].apply(lambda n: n * 1.05 if n > 0 else 0)
pandas_total = df["charged"].sum()

if __name__ == "__main__":
    print(round(pipeline_total, 2))    # 72.45
    print(round(pandas_total, 2))     # 72.45
```